

Python 객체지향 프로그래밍 (OOP) 심화

실용적인 예제와 함께 알아보는 4대 핵심 원칙



캡슐화

Encapsulation



상속

Inheritance



다형성

Polymorphism



추상화

Abstraction

목차



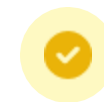
캡슐화 (Encapsulation)

데이터는 어떻게 보호하고 관리하는가?



상속 (Inheritance)

코드를 어떻게 재사용하고 확장하는가?



정리 및 Q&A



다형성 (Polymorphism)

어떻게 유연하고 확장 가능한 코드를 만드는가?



추상화 (Abstraction)

어떻게 명확한 '규격'을 정의하고 협업하는가?

캡슐화 (Encapsulation) - 정의



정의

캡슐화는 데이터(속성)와 그 데이터를 다루는 기능(메서드)을 하나로 묶어 객체 내부에 보관하고, 외부에서는 객체 내부의 데이터에 직접 접근하는 것을 제한하는 객체지향 프로그래밍의 핵심 원칙입니다.



핵심 아이디어

데이터는 숨기고, 오직 허용된 경로(메서드)로만 접근을 제어합니다. 이를 통해 데이터의 무결성을 보장하고, 객체의 상태를 안전하게 유지하는 것을 목표로 합니다.



목표

데이터의 무결성을 보장하고, 객체의 상태를 안전하게 유지합니다. 객체의 내부 상태에 대한 직접적인 접근을 제한하여, 클래스의 일관성을 유지합니다.



구현 방법

- ``_`` (더블 언더스코어): 파이썬에서 속성명 앞에 ``_``를 붙여 비공개(private) 속성임을 나타냅니다.
- Getter/Setter 메서드: 비공개 속성에 접근하거나 값을 변경할 때 사용하는 공개(public) 메서드입니다.



"캡슐화는 객체를 하나의 캡슐처럼 생각할 수 있습니다. 캡슐 안에는 데이터와 함수들이 함께 들어있으며, 캡슐의 외부에서는 캡슐 안의 내용물을 직접 볼 수 없습니다. 대신, 특별한 '구멍'을 통해만 내용물을 확인하거나 변경할 수 있습니다."

캡슐화 - 예제 코드 (1/3)

</> UserProfile 클래스

```
class UserProfile:
    def __init__(self, username, email):
        self.username = username # 공개 속성
        self.__email = None # 비공개 속성
        self.set_email(email) # Setter를 통해 초기화
```

i 코드 설명

라인 2: **def __init__(self, username, email):**

UserProfile 객체가 생성될 때 자동으로 호출되는 생성자 메서드입니다. `self`는 생성될 객체 자신을 참조합니다.

라인 3: **self.username = username**

username은 공개(public) 속성으로, 객체의 사용자 이름을 저장합니다. 외부에서 직접 접근하고 변경할 수 있습니다.

라인 4: **self.__email = None # 비공개 속성**

__email은 이메일 주소를 저장하는 속성입니다. 속성명 앞에 __를 붙여 파이썬에서 비공개(private) 속성임을 나타냅니다.

💡 캡슐화의 목적



캡슐화의 핵심 목적은 데이터 보호와 유효성 검증입니다. 비공개 속성은 외부에서 직접 접근할 수 없으므로, 객체의 상태를 안전하게 유지할 수 있습니다.

캡슐화 - 예제 코드 (2/3)

</> Python

```
7 def get_email(self):  
8     return self.__email  
10  
11 def set_email(self, new_email):  
12     # 유효성 검증: '@' 문자가 포함되어 있는지 확인  
13     if "@" in new_email:  
14         self.__email = new_email print("이메일이  
15         성공적으로 변경되었습니다.")  
16     else:  
        print("오류: 유효하지 않은 이메일 형식입니다.")
```

캡슐화 - 예제 코드 (3/3)

사용 예

```
user1 = UserProfile("john_doe", "john@example.com")
```

```
user1.set_email("john_doe_invalid_email") # -> 오류: 유효하지 않은 이메일 형식입니다.
```

```
# user1.__email 로 직접 접근 불가!
```

🔑 캡슐화의 효과



비공개 속성(__email

상속 (Inheritance) - 정의

정의

상속은 객체지향 프로그래밍의 핵심 원칙 중 하나로, 기존 클래스(부모 클래스 또는 슈퍼 클래스)의 속성과 메서드를 새로운 클래스(자식 클래스 또는 서브 클래스)가 물려받아 재사용하고 확장하는 메커니즘입니다. 이는 코드의 중복을 줄이고, 소프트웨어의 재사용성을 높이며, 유지보수성을 향상시키는 데 기여합니다.

핵심

기존 코드를 재활용하여 코드 중복을 줄이고, 클래스 간의 논리적인 계층 관계를 형성합니다.

목표

- 코드 재사용
- 확장성 유지
- 보수성

구현

```
class 자식클래스 (부모클래스):  
    super() 함수: 부모 클래스의 메서드 호출
```



상속 - 예제 코드 (1/3)

</> Employee 부모 클래스

```
class Employee:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id
    def display_info(self):
        return f"ID: {self.employee_id}, 이름: {self.name}"
```



부모 클래스의 역할

`Employee` 클래스는 모든 직원이 공통적으로 가지는 기본 정보(이름, 사번)와 해당 정보를 출력하는 기능을 정의합니다. 이 클래스는 부모 클래스로서 자식 클래스에게 자신의 속성과 메서드를 물려줄 수 있습니다.



코드 설명

- **1-2** 라인: `Employee` 클래스를 정의하고, 생성자 `\_\_init\_\_` 메서드를 선언합니다. 이 메서드는 객체가 생성될 때 자동으로 호출되며, `name`과 `employee\_id`를 매개변수로 받습니다.
- **3-4** 라인: 전달받은 `name`과 `employee\_id`를 객체의 속성으로 저장합니다. `self.name`과 `self.employee\_id`는 이 `Employee` 객체의 고유한 이름과 사번을 나타냅니다.
- **6-7** 라인: `display\_info` 메서드를 정의합니다. 이 메서드는 객체의 `employee\_id`와 `name`을 포함하는 형식화된 문자열을 반환하여 직원 정보를 보여줍니다.



부모 클래스는 자식 클래스에게 자신의 속성과 메서드를 물려주지만, 자식 클래스는 필요한 경우 이를 오버라이드하거나 확장할 수 있습니다.

상속 - 예제 코드 (2/3)

FullTimeEmployee 클래스 정의

```
9 class FullTimeEmployee(Employee):
10     def __init__(self, name, employee_id, annual_salary):
11         super().__init__(name, employee_id)
12         self.annual_salary = annual_salary
```

클래스 정의

FullTimeEmployee 클래스는 Employee 클래스를 **상속**받습니다. 이는 FullTimeEmployee가 Employee의 모든 속성과 메서드를 사용할 수 있음을 의미합니다.

생성자 메서드

FullTimeEmployee의 생성자 `__init__` 메서드는 부모 클래스의 `name`, `employee_id` 외에 `annual_salary`라는 추가적인 속성을 가집니다.

`super().__init__(name, employee_id)`

`super()`는 부모 클래스(Employee)를 참조하며, `__init__()`을 통해 부모 클래스의 생성자를 호출합니다. 이는 `name`과 `employee_id` 초기화 로직을 재사용합니다.

`self.annual_salary = annual_salary`

FullTimeEmployee는 부모 클래스에서 물려받은 속성 외에 자신만의 고유한 속성을 추가로 초기화합니다.

상속 - 예제 코드 (3/3)

```
1  # 부모 클래스
2  class Employee:
3      def __init__(self, name, employee_id):
4          self.name = name
5          self.employee_id = employee_id
6
7      def display_info(self):
8          return f"ID: {self.employee_id}, 이름: {self.name}"
9
10 # 자식 클래스
11 class FullTimeEmployee(Employee):
12     def __init__(self, name, employee_id, annual_salary):
13         super().__init__(name, employee_id)
14         self.annual_salary = annual_salary
15
16     def display_info(self): # 메서드 오버라이딩 (확장)
17         base_info = super().display_info()
18         return f"{base_info}, 유형: 정규직"
```



메서드 오버라이딩

`FullTimeEmployee` 클래스는 `Employee` 클래스의 `display\_info` 메서드를 **오버라이딩**하여 기능을 확장합니다.



`super().display\_info()`를 호출하여 부모 클래스의 기능을 재사용하
하 후 추가 정보를 바꿉니다



상속의 실제 효과

- ✓ 코드 중복을 줄이고, 유지보수성을 높입니다.
- ✓ 객체 생성 시 `super().\_\_init\_\_()`로 부모 초기화 로직을 재사용합니다.
- ✓ 자식 클래스는 부모 클래스의 기능을 그대로 유지하면서 자신만의 고유한 기능을 추가할 수 있습니다.

다형성 (Polymorphism) - 정의

정의

"하나의 이름, 여러 형태" 를 의미합니다. 동일한 메서드 호출이 객체의 실제 타입에 따라 다르게 동작할 수 있도록 하는 개념입니다.

핵심 아이디어

동일한 메서드 호출에도 객체의 실제 형태에 따라 다른 동작을 합니다.

목표

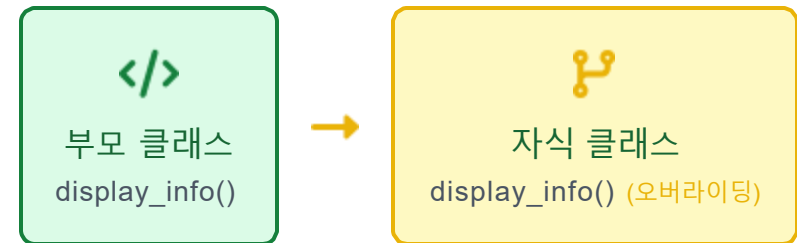
코드를 유연하고 확장 가능하게 만듭니다. (새로운 유형 추가가 쉬워짐)

구현

주로 상속 관계에서 메서드 오버라이딩을 통해 구현됩니다. Python의 덕 타이핑은 다형성을 더욱 유연하게 만듭니다.

다형성의 핵심 특징

- 같은 이름의 메서드가 다른 클래스에서 다르게 구현될 수 있음
- 객체의 실제 타입에 따라 호출되는 메서드가 결정됨
- 코드의 유연성을 높이고, 확장 가능성을 제공



다형성 - 예제 코드 (1/3)

</> 코드 예제

```
# FullTimeEmployee와 PartTimeEmployee는 각각 다른 방식으로
# calculate_monthly_pay() 메서드를 구현하고 있음

employees = [
    FullTimeEmployee("김철수", "EMP001", 60000000),
    PartTimeEmployee("이영희", "EMP002", 15000, 80)
]
```

💡 코드 설명: 객체 리스트 준비

☰ **Line 1-2:** `employees`라는 리스트를 생성합니다. 이 리스트에는 `FullTimeEmployee` 객체와 `PartTimeEmployee` 객체가 함께 포함되어 있습니다.

👤 **Line 3:** `FullTimeEmployee("김철수", "EMP001", 60000000)`: 정규직 직원을 나타내는 객체로, 이름, 사번, 연봉을 가집니다.

👤 **Line 4:** `PartTimeEmployee("이영희", "EMP002", 15000, 80)`: 파트타임 직원을 나타내는 객체로, 이름, 사번, 시급, 근무 시간을 가집니다.

💡 다형성의 핵심: 서로 다른 클래스의 객체들을 하나의 컬렉션에 담을 수 있습니다. 각 객체는 비록 타입은 다르지만, 공통된 인터페이스(여기서는 `calculate\_monthly\_pay()` 메서드)를 제공할 것으로 기대됩니다.

다형성 - 예제 코드 (2/3)

```
def process_payroll(employee_list):  
    for emp in employee_list:  
        # emp가 정규직이든 파트타임이든 상관없이 동일한 코드로 호출  
        # 파이썬이 알아서 객체에 맞는 메서드를 실행  
        monthly_pay = emp.calculate_monthly_pay()  
        print(f"{emp.name}의 월급: {monthly_pay:,.0f}원")
```



함수 정의

process_payroll 함수는 employee_list라는 매개변수를 받습니다. 이 매개변수는 다양한 유형의 직원 객체들을 담고 있는 리스트입니다.



반복문

employee_list에 있는 각 emp(직원 객체)에 대해 반복(루프)을 시작합니다. 이 루프를 통해 함수는 리스트의 모든 직원 객체를 순서대로 처리하게 됩니다.

다형성의 핵심



emp 객체가 FullTimeEmployee이든 PartTimeEmployee이든 관계없이 동일한 방식으로 calculate_monthly_pay() 메서드를 호출할 수 있습니다. 파이썬 런타임이 emp의 실제 타입에 따라 적절한 calculate_monthly_pay() 구현을 찾아 실행합니다.

다형성 - 예제 코드 (3/3)

```
def process_payroll(employee_list):  
    for emp in employee_list:  
        # emp가 정규직이든 파트타임이든 상관없이 동일한 코드로 호출  
        # 파이썬이 알아서 객체에 맞는 메서드를 실행  
        monthly_pay = emp.calculate_monthly_pay()  
        print(f"{emp.name}의 월급: {monthly_pay:,.0f} 원")
```



동일한 인터페이스, 다른 실행

`process\_payroll` 함수는 객체의 실제 타입을 몰라도 동일한 메서드 호출로 일관된 인터페이스를 제공합니다. 파이썬은 객체의 타입에 따라 적절한 `calculate\_monthly\_pay()` 메서드를 자동으로 호출합니다.



코드의 유연성

새로운 직원 유형이 추가되어도 `process\_payroll` 함수는 수정할 필요가 없습니다. 각 새로운 클래스가 `calculate\_monthly\_pay()` 메서드만 구현하면 됩니다. 이는 코드의 확장성을 크게 향상시킵니다.



코드 중복 감소



유지보수 용이



확장 가능

추상화 (Abstraction) - 정의

정의

추상화(Abstraction)는 복잡한 내부 구현을 숨기고, 필수적인 '인터페이스'만 외부에 노출하는 객체지향 프로그래밍의 핵심 원칙입니다. 이는 사용자가 '무엇을' 할 수 있는지에 집중하고, '어떻게' 동작하는지에 대한 세부 사항은 알 필요 없도록 만듭니다.

핵심 아이디어

"어떻게"는 숨기고 "무엇을" 할 수 있는지만 보여줍니다.

목표

시스템의 각 부분이 지켜야 할 '**규격' 또는 '계약**'을 정의하여 안정적인 협업을 가능하게 합니다.

구현

Python에서는 `abc` 모듈의 `ABC` (Abstract Base Class)와 `@abstractmethod` 데코레이터를 사용하여 추상 클래스와 추상 메서드를 정의합니다.



추상화 - 예제 코드 (1/3)

</> Python

```
1 from abc import ABC, abstractmethod
3 # 추상 클래스 (설계도, 규격)
4 class NotificationService(ABC):
5     @abstractmethod
6     def send(self, user_id, message):
7     pass # 내용은 없고, 이름과 파라미터만 정의
```

I

abc 모듈 импорт

Python의 추상화 기능을 사용하기 위해 `abc`

추상화 - 예제 코드 (2/3)

```
10 class EmailNotification(NotificationService):  
11     def send(self, user_id, message): # 반드시 구현해야 함  
12     print(f[Email] {user_id}에게 발송: {message})
```

상속 관계

`EmailNotification` 클래스는 `NotificationService` 추상 클래스를 상속받습니다. 이는 `EmailNotification`이 `NotificationService`가 정의한 모든 추상 메서드를 구현해야 함을 의미합니다.

메서드 구현

`NotificationService`의 추상 메서드였던 `send`를 `EmailNotification` 클래스에서 구체적으로 구현합니다. 이 메서드는 `user\_id`와 `message`를 받아 이메일 형식의 알림 메시지를 출력합니다.

구현의 의미

이 구현은 `EmailNotification` 클래스가 "어떻게" 알림을 보내는지에 대한 구체적인 방법을 제공합니다. 이렇게 함으로써 추상 클래스의 "규격"을 따르면서 각 구현 클래스마다 다른 방식으로 알림을 보내는 것이 가능해집니다.

추상화 - 예제 코드 (3/3)

```
# Line 14                                class "SMSNotification"
class SMSNotification(NotificationService):
# Line 15
    def send(self, user_id, message): # 반드시 구현해야 함
# Line 16
print(f"[SMS] {user_id}에게 발송: {message}")
```



SMSNotification 구현 클래스

`SMSNotification` 클래스는 `NotificationService` 추상 클래스의 `send` 메서드를 구현합니다. 이 클래스는 SMS 알림을 보내는 구체적인 방식을 제공합니다.



이 클래스는 `EmailNotification`과 마찬가지로 `NotificationService` 인터페이스를 따르지만, 알림 방식이 다르게 구현되어 있습니다.

공통 인터페이스의 이점

- ✓ 사용자는 `NotificationService`의 `send` 메서드만 알고 있으면 됩니다. 구현细节은 클래스마다 다를 수 있습니다.
- ✓ 새로운 알림 방식이 추가되어도, 기존 코드를 수정하지 않고도 쉽게 확장할 수 있습니다.
- ✓ 호출하는 쪽에서는 구현 클래스의 세부사항을 알 필요 없이, 인터페이스에 의존할 수 있습니다.

추상화 - 활용 예시

</> 추상 클래스 활용 예

```
# 추상 클래스 (설계도, 규격)
class NotificationService(ABC):
    @abstractmethod
    def send(self, user_id, message):
        pass # 내용은 없고, 이름과 파라미터만 정의

# 실제 구현 클래스 - Email
class EmailNotification(NotificationService):
    def send(self, user_id, message): # 반드시 구현해야 함
        print(f"[Email] {user_id}에게 발송: {message}")

# 실제 구현 클래스 - SMS
class SMSNotification(NotificationService):
    def send(self, user_id, message): # 반드시 구현해야 함
        print(f"[SMS] {user_id}에게 발송: {message}")
```

💡 추상화의 장점

- ✓ **규격 정의**
NotificationService는 모든 알림 클래스가 send() 메서드를 갖도록 강제합니다.
- ✓ **구현 의존성 감소**
사용하는 쪽에서는 send() 메서드가 있다는 사실만 믿고 호출할 수 있습니다.
- ✓ **확장 용이성**
새로운 알림 방식이 추가되어도, 기존 코드 수정 없이 시스템에 통합할 수 있습니다.
- ✓ **협업 용이성**
팀원 간에 "이 인터페이스를 따라 구현하면 된다"라는 명확한 계약을 제공합니다.

캡슐화 - 심화 예제



데이터 유효성 검증 패턴

Setter 메서드에서 데이터의 유효성을 검증하는 고급 패턴입니다. 이메일 형식 검증 예제에서처럼, setter 메서드는 데이터의 무결성을 보장합니다.

```
def set_email(self, new_email):  
    if "@" in new_email:  
        self.__email = new_email  
    else:  
        raise ValueError("유효하지 않은 이메일 형식")
```



접근 제어 패턴

속성에 대한 접근을 제어하는 방식을 다양화하는 패턴입니다. Python에서는 속성 이름에 따라 접근 가능성을 결정할 수 있습니다.

- ◆ 공개 속성: `self.name`

상속 - 심화 예제

파이썬은 다중 상속(multiple inheritance)을 지원하며, 이는 코드 재사용의 유연성을 크게 높입니다. 믹스인(mixin) 패턴은 특정 기능을 여러 클래스에 쉽게 추가할 수 있게 해줍니다.

🔑 다중 상속

```
# 부모 클래스 A
class A:
    def method_a(self):
        return "A의 메서드"

# 부모 클래스 B
class B:
    def method_b(self):
```

C 클래스는 A와 B의 모든 메서드를 동시에 사용할 수 있습니다.

💡 이점

- 코드 중복을 줄이고, 소프트웨어의 재사용성을 높입니다.
- 클래스 간의 논리적인 계층 관계를 형성합니다.
- 유지보수성을 향상시키며, 한 곳에 공통 로직을 모아 관리합니다.

🔗 믹스인 패턴

```
# 믹스인 클래스 - Email 기능
class EmailMixin:
    def send_email(self, message):
        return f"[이메일] {message}"

# 믹스인 클래스 - SMS 기능
class SMSMixin:
    def send_sms(self, message):
```

NotificationService는 EmailMixin과 SMSMixin의 기능을 모두 포함합니다.

다형성 - 심화 예제

덕 타이핑 (Duck Typing)

파이썬은 "오리처럼 걷는 것이 오리다"는 개념을 따르는 덕 타이핑을 통해 다형성을 구현합니다. 객체의 실제 타입보다는 객체가 제공하는 메서드와 속성에 초점을 맞춥니다.

상속 기반 다형성

파이썬에서 다형성은 주로 상속 관계에서 메서드 오버라이딩을 통해 구현됩니다. 자식 클래스는 부모 클래스의 메서드를 재정의(오버라이드)하여 필요한 부분만 확장합니다.

유연한 타입 검사

파이썬은 런타임에 객체의 타입을 확인하지 않고, 필요한 메서드나 속성의 존재만 확인합니다. 이는 동일한 인터페이스를 제공하는 객체들을 서로 다르게 처리하지 않습니다.

파이썬 다형성의 특성

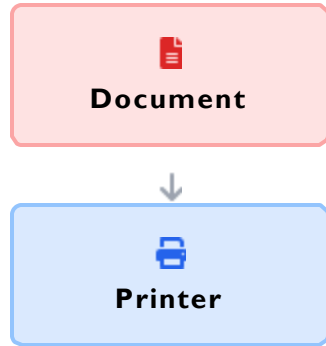
- ✓ 코드 유연성: 동일한 메서드 호출이 객체의 실제 타입에 따라 다르게 동작할 수 있습니다.
- ✓ 내부 구현 숨김: 객체의 실제 구현细节은 중요하지 않으며, 필요한 인터페이스만 제공하면 됩니다.
- ✓ 확장 용이성: 새로운 클래스를 추가하더라도 기존 코드를 수정할 필요가 없습니다.
- ✓ 동적 바인딩: 메서드 호출 시점에 실제 객체의 타입을 알 수 있으며, 이에 따라 적절한 메서드가 실행됩니다.

추상화 - 심화 예제

인터페이스 분리 원칙

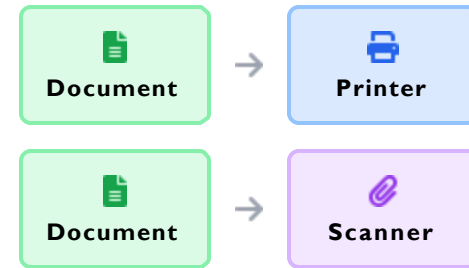
추상화의 심화된 적용으로, 클라이언트가 사용하지 않는 인터페이스에 의존하지 않도록 하는 설계 원칙입니다.

✖ 잘못된 설계



Printer 클래스가 Document 인터페이스의 모든 메서드를 구현해야 함
→ 불필요한 의존성!

✔ 올바른 설계



각 클래스는 필요한 인터페이스만 구현
→ 인터페이스 분리!

인터페이스 분리의 이점

- 클라이언트 코드의 변경에 영향을 받지 않음
- 시스템의 유지보수가 용이해짐
- 코드의 의존성 그래프가 더 간결해짐
- 새로운 기능 추가가 쉬워짐

객체지향 설계 원칙 - SOLID

SOLID는 객체지향 설계의 5가지 핵심 원칙으로, 소프트웨어의 유지보수성, 확장성, 이해성을 높입니다. 이 원칙들은 OOP의 4대 핵심 원칙과 밀접한 관련이 있습니다.



SRP

Single Responsibility

클래스 하나당 책임 하나



OCP

Open/Closed

확장에는 열려있으나 수정에는 닫혀 있음



LSP

Liskov Substitution

자식은 부모로 대체할 수 있어야 함



ISP

Interface Segregation

특정 클라이언트를 위한 인터페이스



DIP

Dependency Inversion

의존성 주입과 역전

SOLID와 OOP 4대 원칙의 관계



캡슐화 (Encapsulation)

데이터 보호와 유효성 검증

SRP, OCP, LSP



다형성 (Polymorphism)

동일한 요청, 다른 실행

LSP, ISP, DIP



상속 (Inheritance)

코드 재사용과 확장

OCP, LSP, ISP



추상화 (Abstraction)

필수 기능의 규격화

ISP, DIP

객체지향 프로그래밍의 한계

⚠ 객체지향 접근 방식의 한계점

- ❌ 복잡성 증가: 클래스와 객체의 관계가 복잡해지면 코드 이해와 유지보수가 어려워짐
- ❌ 오버헤드: 객체 생성, 메서드 호출 등으로 인한 성능 저하
- ❌ 상속의 남용: 깊은 상속 계층구조로 인한 코드의 가독성 감소
- ❌ 데이터 무결성: 캡슐화로 인한 데이터 접근 제한이 너무 많을 수 있음

⚖ 함수형 프로그래밍과의 비교



객체지향

- ✓ 상태 변경에 의한 부수 효과
- ✓ 객체의 생애 주기 관리
- ✓ 다형성과 상속



함수형

- ✓ 불변성과 순수 함수
- ✓ 선언형 코드 병
- ✓ 련 처리에 적합

💡 보완점

OOP와 함수형 프로그래밍의 장점을 결합한 '객체지향 함수형 프로그래밍' 접근 방식이 가능합니다. 예를 들어, Python의 함수는 일급 객체이므로 함수 자체를 객체로 취급하거나, 클래스 메서드로 정의할 수 있습니다.

파이썬 특유의 객체지향 특성



동적 타이핑 (Dynamic Typing)

변수의 타입이 런타임에 결정되므로, 객체의 실제 타입에 따라 다른 메서드를 호출할 수 있습니다. 이는 다형성을 더욱 강화합니다.



덕 타이핑 (Duck Typing)

객체의 실제 클래스보다는 특정 메서드나 속성을 가지고 있는지에 초점을 맞춥니다. "만약 오리처럼 걷는다면 그것은 오리다"는 개념을 프로그래밍에 적용한 것입니다.



비공개 속성 (Private Attributes)

속성명 앞에 ``__`` (더블 언더스코어)를 붙여 비공개(private) 속성으로 만듭니다. 파이썬은 이름 망글링을 통해 외부에서의 직접 접근을 어렵게 만듭니다.



`super()` 함수

자식 클래스에서 부모 클래스의 메서드를 호출할 때 사용합니다. 특히 ``__init__`` 메서드에서 부모의 초기화 로직을 재사용할 때 필수적입니다.



abc 모듈

추상 클래스(ABC)와 ``@abstractmethod`` 데코레이터를 제공하여 추상화를 구현합니다. 이는 인터페이스의 역할을 합니다.



연유한 상속 (Flexible Inheritance)

파이썬은 다중 상속을 지원하고, 클래스는 런타임에 동적으로 수정될 수 있습니다. 이는 상속 관계를 매우 유연하게 만듭니다.

실전 응용 - 디자인 패턴

디자인 패턴은 객체지향 프로그래밍의 4대 원칙을 결합하여 특정 문제에 대한 유연하고 재사용 가능한 솔루션을 제공합니다.

캡슐화 패턴

- ◆ 프록시 패턴 (**Proxy Pattern**): 객체에 대한 접근을 제어하고, 필요한 경우에만 객체를 생성합니다.
- ◆ 데코레이터 패턴 (**Decorator Pattern**): 객체에 책임을 동적으로 추가하거나 수정합니다.

상속 패턴

- ◆ 템플릿 메서드 (**Template Method**): 알고리즘의 일부 단계를 서브클래스에서 재정의하도록 합니다.
- ◆ 스트레티지 패턴 (**Strategy Pattern**): 알고리즘을 클래스로 캡슐화하고, 객체의 행동을 변경할 수 있게 합니다.

다형성 패턴

- ◆ 팩토리 메서드 (**Factory Method**): 객체 생성 인터페이스를 제공하지만, 어떤 클래스의 인스턴스를 생성할지 결정합니다.
- ◆ 옵저버 패턴 (**Observer Pattern**): 객체의 상태 변경을 다른 객체에게 자동으로 알립니다.

추상화 패턴

- ◆ 빌더 패턴 (**Builder Pattern**): 복잡한 객체를 생성하는 과정을 추상화하고, 생성 단계를 제어합니다.
- ◆ 컴퍼지트 패턴 (**Composite Pattern**): 개별 객체와 복합 객체를 동일한 인터페이스로 처리합니다.

 이러한 디자인 패턴들은 객체지향 프로그래밍의 4대 원칙을 결합하여, 더 유연하고 확장 가능한 코드를 작성하는 데 도움이 됩니다.

객체지향 프로그래밍 테스트 전략

테스트 방법론

 **유닛 테스트 (Unit Testing)**
개별 클래스나 메서드의 기능을 독립적으로 검증합니다. 캡슐화된 속성과 메서드의 동작을 확인합니다.

 **통합 테스트 (Integration Testing)**
클래스 간의 상호작용을 테스트합니다. 특히 상속 관계에서 부모-자식 클래스들의 協同 동작을 검증합니다.

 **시스템 테스트 (System Testing)**
전체 시스템의 일관된 동작을 검증합니다. 모든 객체들이 예상대로 상호작용하는지 확인합니다.

모킹 전략

무엇이 모킹인가?

모킹(Mocking)은 테스트 중인 객체의 의존성을 가짜(mock) 객체로 대체하는 테스트 기술입니다.

모킹의 목적

- 외부 의존성을 제거하여 테스트의 일관성을 유지합니다.
- 예외 상황을 쉽게 시뮬레이트할 수 있습니다.
- 코드의 특정 부분에 대한 테스트를 쉽게 수행할 수 있습니다.

파이썬 모킹 도구

unittest.mock

파이썬 표준 라이브러리

pytest-mock

pytest 확장 플러그인

객체지향 프로그래밍의 성능 고려사항

⚠ 성능에 미치는 영향

- ⚙ 메서드 호출 오버헤드
메서드 호출 시점에 객체의 메서드 테이블을 검색하는 데 시간이 소요됩니다.
- 📊 메모리 소비 증가
객체마다 메서드 테이블에 대한 참조와 인스턴스 변수 저장소가 필요합니다.
- 🔗 상속 계층의 복잡성
깊은 상속 계층은 메서드 호출 시간을 증가시키며, 이해하기도 어렵습니다.
- 🔄 다형성의 오버헤드
런타임에 실제 호출될 메서드를 결정하는 데 시간이 소요됩니다.

⚡ 최적화 전략

- 🌱 컴포지션 대신 상속
상속보다는 합성(composition)을 사용하여 클래스의 크기를 줄이고, 필요한 기능만 포함시킵니다.
- 👥 객체 그래프 최적화
객체 간 참조 관계를 최소화하고, 불필요한 객체 생성을 피합니다.
- 📄 메서드 인라인
자주 호출되고 로직이 간단한 메서드를 인라인으로 작성하여 메서드 호출 오버헤드를 줄입니다.
- 📈 프로파일링과 벤치마킹
성능 병점을 식별하고, 최적화의 효과를 측정하기 위해 프로파일링 도구를 사용합니다.

정리: 객체지향 프로그래밍의 힘

캡슐화 (Encapsulation)

핵심 아이디어: 데이터 보호와 유효성 검증

이점: 안정성, 데이터 무결성

다형성 (Polymorphism)

핵심 아이디어: 동일한 요청, 다른 실행

이점: 유연성, 확장 용이성

상속 (Inheritance)

핵심 아이디어: 코드 재사용과 확장

이점: 효율성, 유지보수 용이

추상화 (Abstraction)

핵심 아이디어: 필수 기능의 규격화

이점: 단순성, 원활한 협업

 결론: 이 원칙들을 잘 활용하면 안정적이고, 유연하며, 관리하기 쉬운 소프트웨어를 만들 수 있습니다.

Q&A



코드 예제에 대한 질문

오늘 살펴본 캡슐화, 상속, 다형성, 추상화 예제 코드에 대해 궁금한 점이 있으신가요?

개념 확장

이 4가지 원칙을 어떻게 실제 프로젝트에 적용할 수 있을까요?

도구와 기술

파이썬 외에도 다른 언어에서 객체지향 프로그래밍은 어떻게 다루나요?

협업과 유지보수

객체지향 원칙이 팀 개발과 소프트웨어 유지보수에 어떻게 도움이 되나요?